Documentación de Base de Datos REGIVA Costa Rica

Índice

Documentación de Base de Datos REGIVA Costa Rica **1**
- Índice 1
- Visión General 3
  - Arquitectura Multi-Tenant 3
  - Principios de Diseño 3
- Diagrama Lógico 4
- Descripción de Módulos 5
  - 1. Módulo Multi-Tenant (Core) 5
    - tenants 5
    - users 5
    - tenant_users 5
  - 2. Módulo de Entidades de Negocio 5
    - customers 5
    - suppliers 6
    - products 6
  - 3. Módulo de Documentos Electrónicos 6
    - electronic_documents 6
    - document_lines 7
    - document_taxes 7
    - document_xml_storage 7
    - payment_receipts 8
  - 4. Módulo de Inteligencia Artificial 8
    - cash_flow_projections 8
    - credit_scores 8
    - financial_metrics 9
  - 5. Módulo de Configuración y Seguridad 9
    - hacienda_credentials 9
    - system_configuration 9
  - 6. Módulo de Auditoría 10
    - activity_logs 10
- Relaciones Clave 10
  - Diagrama Entidad-Relación (Resumen) 10
- Índices y Performance 10
  - Índices Estratégicos Implementados 11
    - Alto Impacto (Queries Frecuentes) 11
    - Performance de Joins 11

Análisis e IA 11
Estrategia de Particionamiento (Futuro) 11
Integración con ASP.NET MVC 12
Arquitectura por Capas 12
1. Capa de Presentación (UI - Razor Views) 12
2. Capa de Controladores (MVC) 12
3. Capa de Lógica de Negocio (Services) 12
4. Capa de Acceso a Datos (Repository) 13
Configuración de Npgsql 13
Soft Delete Pattern 14
Estrategia de Escalabilidad 14
Fase 1: MVP (0-1,000 empresas) 14
Fase 2: Growth (1,000-10,000 empresas) 15
Fase 3: Scale (10,000+ empresas) 15
Monitoreo de Performance 15
Seguridad y Compliance 15
Encriptación 15
GDPR / Ley 8968 (Costa Rica) 15
Backup Strategy 15
Casos de Uso Frecuentes 16
1. Crear Factura Completa 16
2. Dashboard Cuentas por Cobrar 16
3. Calcular DSO Real (con REP) 16
4. Proyección de Flujo de Caja (próximos 30 días) 16
Siguiente Pasos 17
Soporte y Documentación Adicional 17

Visión General

Arquitectura Multi-Tenant

El sistema soporta **múltiples empresas (tenants)** en una única base de datos mediante discriminación por `tenant_id`. Cada empresa cliente del SaaS tiene:

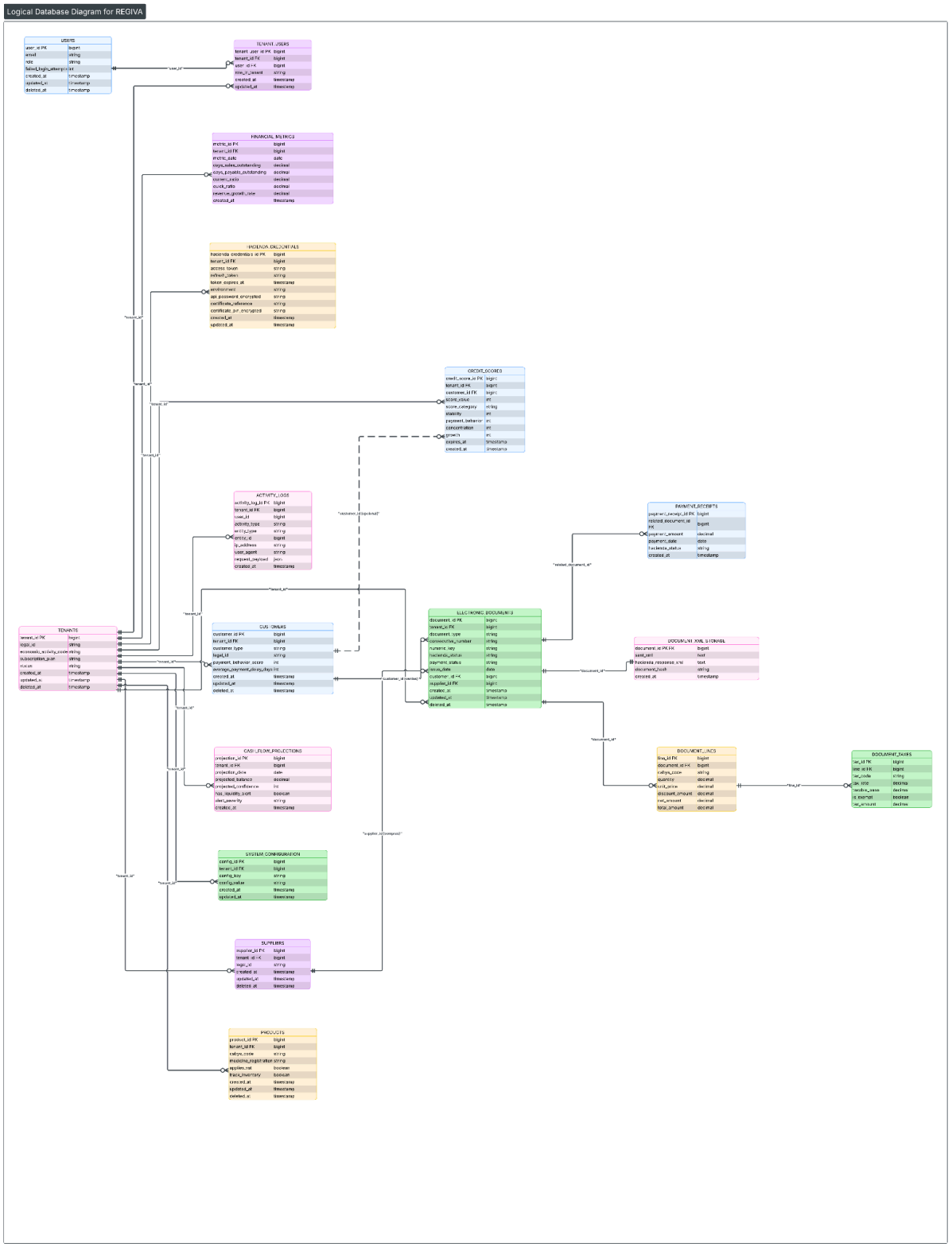
- Su propio conjunto de datos aislado lógicamente
- Usuarios propios (con roles específicos)
- Configuración independiente
- Facturación y análisis separados

Principios de Diseño

- **Normalización 3FN**: Reducción de redundancia
- **Soft Delete**: `deleted_at` permite recuperación
- **Auditoría completa**: `created_at`, `updated_at`, `created_by`, `updated_by`
- **Seguridad**: Credenciales cifradas, referencias a vault externos
- **Escalabilidad**: Índices estratégicos, particionamiento futuro

---

Diagrama Lógico



Descripción de Módulos

1. Módulo Multi-Tenant (Core)

`tenants`

Propósito: Empresas clientes del sistema SaaS.

Campos Clave:

- `legal_id`: Cédula jurídica única (CR)
- `economic_activity_code`: CAE para Hacienda
- `subscription_plan`: basic/professional/enterprise
- `status`: Gestión del ciclo de vida del cliente

Uso: Cada query debe filtrar por `tenant_id` para aislar datos.

`users`

Propósito: Usuarios individuales del sistema.

Campos Clave:

- `email`: Único en el sistema
- `role`: Rol global (super_admin puede acceder a todo)
- `failed_login_attempts`: Seguridad anti-fuerza bruta

Uso: Un usuario puede pertenecer a múltiples tenants.

`tenant_users`

Propósito: Relación N:M entre usuarios y empresas.

Campos Clave:

- `role_in_tenant`: Rol específico dentro de la empresa (owner, admin, etc.)
- Permite que un contador sea "admin" en empresa A y "viewer" en empresa B

---

2. Módulo de Entidades de Negocio

`customers`

Propósito: Clientes de cada empresa tenant.

Campos Clave:

- `customer_type`: individual/business/foreign
- `legal_id`: Único por tenant (cédula/DIMEX)
- `payment_behavior_score`: Calculado por IA
- `average_payment_delay_days`: Métrica histórica

Uso: Base para análisis de cartera y scoring crediticio.

`suppliers`

Propósito: Proveedores de cada empresa tenant.

Similar a customers pero enfocado en compras.

`products`

Propósito: Catálogo de productos/servicios.

Campos Clave:

- `cabys_code`: **OBLIGATORIO** (13 dígitos, estándar CR)
- `medicine_registration`: Para sector salud (v4.4)
- `applies_vat`: Manejo automático de impuestos
- `track_inventory`: Flag para control de stock

Uso: Cada línea de factura referencia un producto.

---

3. Módulo de Documentos Electrónicos

`electronic_documents`

Propósito: **Tabla central** del sistema. Almacena todas las facturas, notas de crédito, tiquetes.

Campos Clave:

- `document_type`: Código Hacienda ('01'=factura, '03'=NC, etc.)
- `consecutive_number`: Numeración secuencial por tipo
- `numeric_key`: Clave única de 50 dígitos (Hacienda)
- `hacienda_status`: Estado del documento en backend fiscal
- `payment_status`: Estado de cobro (pending/paid/overdue)
- `receiver_type` + `customer_id/supplier_id`: Polimorfismo para ventas/compras

Relaciones:

- **1:N** con `document_lines` (detalles)

- **1:1** con `document_xml_storage` (XML completo)
- **1:N** con `payment_receipts` (REP para pagos parciales)

Uso: Query principal para dashboard financiero.

`document_lines`

Propósito: Líneas individuales de cada documento.

Campos Clave:

- `cabys_code`: Clasificación fiscal por item
- `quantity`, `unit_price`, `discount_amount`
- `net_amount`: Subtotal después de descuento
- `total_amount`: Incluyendo impuestos

Relaciones:

- **N:1** con `electronic_documents`
- **1:N** con `document_taxes`

`document_taxes`

Propósito: Impuestos aplicados por línea.

Campos Clave:

- `tax_code`: '01'=IVA, '02'=Selectivo, etc.
- `tax_rate`: Porcentaje
- `taxable_base`: Base sobre la que se calcula
- `is_exempt`: Exoneraciones especiales

Uso: Permite múltiples impuestos por línea (ej: IVA + selectivo de consumo).

`document_xml_storage`

Propósito: Almacena **XML completos** (texto grande).

⚠️ **IMPORTANTE**:

- Esta tabla crece rápidamente
- Considerar estrategia de archivado (6+ meses a almacenamiento frío)
- `TEXT` permite XMLs de 1GB+ pero evaluar JSONB para queries

Campos Clave:

- `sent_xml`: XML firmado enviado a Hacienda
- `hacienda_response_xml`: Respuesta oficial

- `document_hash`: SHA-256 para integridad

`payment_receipts`

Propósito: Recibos electrónicos de pago (REP) - **Nuevo en v4.4**.

Campos Clave:

- `related_document_id`: FK a la factura pagada
- `payment_amount`: Monto de este pago (permite parciales)
- `payment_date`: **Clave para calcular DSO real**

Uso: Correlacionar `issue_date` de factura con `payment_date` de REP = días reales de cobro.

---

4. Módulo de Inteligencia Artificial

`cash_flow_projections`

Propósito: Predicciones de flujo de caja generadas por modelo LSTM.

Campos Clave:

- `projection_date`: Fecha futura proyectada
- `projected_balance`: Saldo estimado
- `projected_*_confidence`: Confianza del modelo (0-100%)
- `has_liquidity_alert`: Flag para alertas críticas

Uso:

- Dashboard principal para CFO virtual
- Trigger alertas WhatsApp si `alert_severity = 'critical'`

`credit_scores`

Propósito: Score crediticio alternativo (sin bureaus tradicionales).

Campos Clave:

- `score_value`: 0-100
- `score_category`: excellent/good/fair/poor/bad
- Componentes: stability, payment_behavior, concentration, growth
- `expires_at`: Los scores deben recalcularse periódicamente

Uso:

- Pre-calificación para factoreo
- Comercialización a bancos/fintech

`financial_metrics`

Propósito: KPIs financieros calculados periódicamente.

Campos Clave:

- `days_sales_outstanding` (DSO): Métrica crítica de cobro
- `days_payable_outstanding` (DPO): Días promedio de pago
- `current_ratio`, `quick_ratio`: Liquidez
- `revenue_growth_rate`: Crecimiento

Uso: Reportería ejecutiva, alimenta modelos de IA.

---

5. Módulo de Configuración y Seguridad

`hacienda_credentials`

Propósito: Gestión de autenticación OAuth con Hacienda.

⚠️ **SEGURIDAD CRÍTICA**:

- `api_password_encrypted`: **NUNCA** plain text
- `certificate_reference`: Path a HashiCorp Vault / AWS KMS
- `certificate_pin_encrypted`: PIN del .p12 cifrado

Campos Clave:

- `access_token`, `refresh_token`: OAuth 2.0
- `token_expires_at`: Renovación automática
- `environment`: sandbox/production

Uso: Backend debe verificar expiración antes de cada llamada API.

`system_configuration`

Propósito: Configuración flexible key-value.

Ejemplos:

- URLs de API Hacienda
- Tasas de impuestos
- Límites de plan free
- Feature flags para A/B testing

---

6. Módulo de Auditoría

`activity_logs`

Propósito: Log completo de actividades (compliance, debugging).

Campos Clave:

- `activity_type`: 'login', 'document_created', 'export_data'
- `entity_type`, `entity_id`: Trazabilidad
- `ip_address`, `user_agent`: Seguridad
- `request_payload`: JSONB para análisis forense

Uso:

- Auditoría externa (ISO 27001)
- Debugging de errores en producción

Relaciones Clave

Diagrama Entidad-Relación (Resumen)

tenant (1) ──< (N) customers
tenant (1) ──< (N) suppliers
tenant (1) ──< (N) products
tenant (1) ──< (N) electronic_documents

customer (1) ──< (N) electronic_documents [ventas]
supplier (1) ──< (N) electronic_documents [compras]

electronic_document (1) ──< (N) document_lines
document_line (1) ──< (N) document_taxes

electronic_document (1) ── (1) document_xml_storage
electronic_document (1) ──< (N) payment_receipts

tenant (1) ──< (N) cash_flow_projections
tenant (1) ──< (N) credit_scores
customer (0..1) ──< (N) credit_scores

Índices y Performance

Índices Estratégicos Implementados

Alto Impacto (Queries Frecuentes)

-- Dashboard principal: documentos recientes
idx_documents_tenant + idx_documents_issue_date

-- Búsqueda de facturas por cliente
idx_documents_customer

-- Cuentas por cobrar pendientes
idx_documents_payment_status

-- Búsqueda por clave numérica (integración Hacienda)
idx_documents_numeric_key (UNIQUE)

Performance de Joins

-- Relación documentos ↔ líneas (muy frecuente)
idx_document_lines_document

-- Relación líneas ↔ impuestos
idx_document_taxes_line

-- XML storage (1:1 lookup)
idx_xml_storage_document (UNIQUE)

Análisis e IA

-- Proyecciones de flujo de caja
idx_cashflow_tenant_date (composite)

-- Scores crediticios
idx_credit_scores_tenant
idx_credit_scores_customer

Estrategia de Particionamiento (Futuro)

Cuando alcanzar:

- `electronic_documents`: 10M+ registros
- `activity_logs`: 100M+ registros

Técnica: Particionamiento por rango de fechas (mensual)

```
-- Ejemplo (no ejecutar ahora):
CREATE TABLE electronic_documents_2025_01 PARTITION OF electronic_documents
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

Integración con ASP.NET MVC

Arquitectura por Capas

1. Capa de Presentación (UI - Razor Views)

```
Views/
├── Documents/
│   ├── Index.cshtml
│   ├── Create.cshtml
│   └── Details.cshtml
├── Customers/
└── Dashboard/
```

2. Capa de Controladores (MVC)

```
public class DocumentsController : Controller
{
    private readonly IDocumentService _documentService;
    private readonly ITenantContext _tenantContext;

    public DocumentsController(IDocumentService service, ITenantContext context)
    {
        _documentService = service;
        _tenantContext = context;
    }

    [Authorize]
    public ActionResult Index()
    {
        int tenantId = _tenantContext.GetCurrentTenantId();
        var documents = _documentService.GetDocumentsByTenant(tenantId);
        return View(documents);
    }
}
```

3. Capa de Lógica de Negocio (Services)

```
public interface IDocumentService
{
    IEnumerable<ElectronicDocument> GetDocumentsByTenant(int tenantId);
    ElectronicDocument CreateInvoice(InvoiceDto dto);
    void SendToHacienda(int documentId);
}
```

```
public class DocumentService : IDocumentService
{
    private readonly IDocumentRepository _repository;
    private readonly IHaciendaApiClient _haciendaClient;

    public ElectronicDocument CreateInvoice(InvoiceDto dto)
    {
        // Validaciones
        // Cálculos de impuestos
        // Generación de XML
        var document = _repository.Create(dto);
        return document;
    }
}
```

4. Capa de Acceso a Datos (Repository)

```
public interface IDocumentRepository
{
    IEnumerable<ElectronicDocument> GetByTenant(int tenantId);
    ElectronicDocument GetById(long documentId);
    ElectronicDocument Create(ElectronicDocument document);
    void Update(ElectronicDocument document);
    void SoftDelete(long documentId);
}

// Implementación con Dapper (recomendado para performance)
public class DocumentRepository : IDocumentRepository
{
    private readonly IDbConnection _db;

    public IEnumerable<ElectronicDocument> GetByTenant(int tenantId)
    {
        string sql = @"
            SELECT d.*, c.legal_name as customer_name
            FROM electronic_documents d
            LEFT JOIN customers c ON d.customer_id = c.customer_id
            WHERE d.tenant_id = @TenantId
              AND d.deleted_at IS NULL
            ORDER BY d.issue_date DESC
            LIMIT 100";

        return _db.Query<ElectronicDocument>(sql, new { TenantId = tenantId });
    }
}
```

Configuración de Npgsql

Web.config / App.config:

```
<connectionStrings>
  <add name="REGIVAConnection"

connectionString="Host=localhost;Database=regiva_cr;Username=regiva_user;Password=secret;Pooling=true;MinPoolSize=5;MaxPoolSize=100;"
      providerName="Npgsql" />
</connectionStrings>
```

Startup Configuration:

```
// Registrar Npgsql
NpgsqlConnection.GlobalTypeMapper.EnableDynamicJson();

// Dependency Injection
services.AddScoped<IDbConnection>(sp =>
   new NpgsqlConnection(Configuration.GetConnectionString("REGIVAConnection")));
```

Soft Delete Pattern

En Repository:

```
public void SoftDelete(long documentId)
{
   string sql = "UPDATE electronic_documents SET deleted_at = @Now WHERE document_id = @Id";
   _db.Execute(sql, new { Now = DateTime.UtcNow, Id = documentId });
}
```

En Queries (siempre filtrar):

```
WHERE deleted_at IS NULL
```

Estrategia de Escalabilidad

Fase 1: MVP (0-1,000 empresas)

- Base de datos única
- Índices implementados
- Queries optimizadas con EXPLAIN ANALYZE
- Connection pooling (Npgsql)

Fase 2: Growth (1,000-10,000 empresas)

- **Read Replicas**: Queries de análisis/reportes a réplicas
- **Particionamiento**: `electronic_documents` por fecha
- **Archivado**: XML > 12 meses a S3/storage frío
- **Caché**: Redis para métricas frecuentes

Fase 3: Scale (10,000+ empresas)

- **Sharding por tenant_id**: Empresas grandes en shards dedicados
- **CQRS**: Separar escritura (transaccional) de lectura (análisis)
- **Event Sourcing**: Para auditoría y reconstrucción de estado

Monitoreo de Performance

```
-- Queries lentas (> 200ms)
SELECT query, mean_exec_time, calls
FROM pg_stat_statements
WHERE mean_exec_time > 200
ORDER BY mean_exec_time DESC
LIMIT 10;

-- Índices no usados (candidatos a eliminación)
SELECT schemaname, tablename, indexname
FROM pg_stat_user_indexes
WHERE idx_scan = 0;
```

Seguridad y Compliance

Encriptación

1. **At Rest**: PostgreSQL TDE (Transparent Data Encryption)
2. **In Transit**: SSL/TLS obligatorio en conexiones
3. **Application Level**:
    - Passwords: BCrypt (ASP.NET Identity)
    - Datos sensibles: AES-256

GDPR / Ley 8968 (Costa Rica)

- Campo `deleted_at` permite "derecho al olvido"
- Logs de consentimiento en `activity_logs`
- Exportación de datos: queries por `tenant_id`

Backup Strategy

```
# Backup diario (cron)
pg_dump -U regiva_user -Fc regiva_cr > backup_$(date +%Y%m%d).dump

# Restore
pg_restore -U regiva_user -d regiva_cr backup_20250101.dump
```

Casos de Uso Frecuentes

1. Crear Factura Completa

```
BEGIN;
-- 1. Insertar documento
INSERT INTO electronic_documents (tenant_id, document_type, ...) VALUES (...);
-- 2. Insertar líneas
INSERT INTO document_lines (document_id, line_number, ...) VALUES (...);
-- 3. Insertar impuestos
INSERT INTO document_taxes (line_id, tax_code, ...) VALUES (...);
-- 4. Guardar XML
INSERT INTO document_xml_storage (document_id, sent_xml) VALUES (...);
COMMIT;
```

2. Dashboard Cuentas por Cobrar

```
SELECT * FROM v_accounts_receivable
WHERE tenant_id = 123
  AND status_category = 'overdue'
ORDER BY days_overdue DESC;
```

3. Calcular DSO Real (con REP)

```
SELECT
    d.document_id,
    d.issue_date,
    pr.payment_date,
    EXTRACT(DAY FROM (pr.payment_date - d.issue_date)) as days_to_payment
FROM electronic_documents d
INNER JOIN payment_receipts pr ON pr.related_document_id = d.document_id
WHERE d.tenant_id = 123
  AND d.document_type = '01'
  AND pr.hacienda_status = 'accepted';
```

4. Proyección de Flujo de Caja (próximos 30 días)

```
SELECT
```

```
    projection_date,
    projected_balance,
    has_liquidity_alert
FROM cash_flow_projections
WHERE tenant_id = 123
  AND projection_date BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL
'30 days'
ORDER BY projection_date;
```

Siguiente Pasos

1. **Ejecutar el Script SQL** en PostgreSQL

Crear Usuario de Aplicación:

```
 CREATE USER regiva_app WITH PASSWORD 'secure_password';
GRANT CONNECT ON DATABASE regiva_cr TO regiva_app;
GRANT USAGE ON SCHEMA public TO regiva_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
regiva_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO regiva_app;
```

2.
3. **Implementar Capa de Datos en ASP.NET**

4. **Configurar DBeaver**:

    - Connection: PostgreSQL
    - Host: localhost:5432
    - Database: regiva_cr
    - Visualizar diagramas: ERD (clic derecho en DB)
5. **Poblar Datos de Prueba** (ver archivo seeds adicional)

Soporte y Documentación Adicional

- **PostgreSQL Docs**: https://www.postgresql.org/docs/
- **Npgsql (.NET)**: https://www.npgsql.org/
- **Hacienda CR API**: https://www.hacienda.go.cr/ATV/ComprobanteElectronico/
- **UBL 2.1 Standard**: http://docs.oasis-open.org/ubl/UBL-2.1.html

---

Versión: 1.0
Fecha: Diciembre 2024
Autor: Diseño para Sistema REGIVA Costa Rica
Licencia: Propietario